

Chapter 5: QSAR & ADMET Property Modeling

Chapter 4 closed by promising that DeepChem, introduced there as a framework layer over RDKit, “becomes the more direct, load-bearing tool starting in Chapter 5.” Building this chapter’s hands-on project surfaced a concrete reason to revisit that plan rather than follow it literally — documented honestly in Section 5.5, in the same spirit as Chapter 4’s own note on the live ChEMBL API outage it encountered. What this chapter *does* build directly on Chapter 4: the ETL pipeline’s output — a clean, one-row-per-compound table of standardized structures and measured bioactivities — is exactly the shape of dataset a QSAR model trains on. This chapter turns that table into predictions.

5.1 QSAR Principles

QSAR (Quantitative Structure-Activity Relationship) modeling is the premise that a molecule’s structure determines its biological or physicochemical behavior closely enough that a model fit on (structure, measured property) pairs can predict that property for a new, untested structure. Structure enters the model exactly as Chapter 2 represented it — most commonly as a fingerprint, since a fixed-length bit vector is what a standard supervised learning algorithm expects as input — and the model outputs either a continuous value (**regression**: predict an IC50, a solubility, a clearance rate) or a discrete label (**classification**: predict active/inactive, blocker/non-blocker). This chapter’s hands-on project (Section 5.5) is a classification task.

The field’s methodology has shifted substantially in emphasis, though not in its core premise, over several decades. Classical QSAR — the Hansch-type linear regression analyses that gave the field its name — fit a small number of hand-selected physicochemical descriptors (LogP, molar refractivity, Hammett substituent constants) against a linear model, favoring interpretability: each coefficient has a direct chemical meaning. Modern QSAR, including this chapter’s project, typically fits a non-linear ensemble model (Random Forest, gradient boosted trees) or a deep neural network on a high-dimensional fingerprint or a learned representation, trading some of that per-descriptor interpretability for substantially better predictive accuracy on complex, non-additive structure-activity relationships. Section 5.3 covers three specific non-linear methods in this modern line; Chapter 6 covers the further step of learning the representation itself (graph neural networks) rather than fixing it as a Chapter 2 fingerprint beforehand.

One property of chemical data is easy to overlook coming from generic supervised learning and becomes central to this chapter: training examples are not independent and identically distributed in the way a generic ML textbook assumes. Chemical libraries cluster into families of close structural analogues built on the same synthetic scaffold — dozens of compounds that differ only in a substituent — so two “different” training examples can be nearly redundant. Section 5.4 shows exactly why this matters: it determines whether a train/test split actually measures generalization to new chemical space, or measures something closer to interpolation between near-duplicates.

5.2 ADMET Profiling

Chapter 1 (Section 1.1) established that most clinical drug candidates fail, and that failure is concentrated in efficacy — but efficacy is not the only thing that has to hold up. **ADMET** — Absorption, Distribution, Metabolism, Excretion, and Toxicity — is the standard shorthand for

the pharmacokinetic and safety properties that determine whether an efficacious compound can actually become a viable, safely dosable drug. Van de Waterbeemd and Gifford (2003), in an early and still-influential assessment of the field, framed the core promise and the core difficulty of *in silico* ADMET modeling in the same phrase: computational prediction could move these assessments far earlier in the pipeline than the animal and human studies that traditionally measure them directly — “towards prediction paradise” — but only to the extent the underlying models are trained on data that actually captures the relevant biology, which for many ADMET endpoints has historically been scarcer and noisier than potency data.

This chapter’s hands-on project focuses on one specific, well-studied toxicity endpoint: **hERG channel blockade**. hERG (the human Ether-à-go-go-Related Gene, official gene symbol *KCNH2*) encodes the pore-forming subunit of the Kv11.1 voltage-gated potassium channel, responsible for the rapid delayed-rectifier current (I_{Kr}) that repolarizes the cardiac action potential after each heartbeat. Sanguinetti and Tristani-Firouzi (2006), reviewing the channel’s biology, describe why it is disproportionately liable to off-target drug binding: its pore cavity is unusually large and lined with aromatic residues, allowing a structurally diverse range of otherwise unrelated drugs to bind and block it, in a way most other ion channels do not permit. Blocking hERG prolongs the cardiac action potential — measured clinically as QT-interval prolongation — which increases the risk of *torsade de pointes*, a potentially fatal ventricular arrhythmia. Because this liability is (a) common across unrelated drug classes and (b) potentially fatal, hERG screening is now a routine, early safety-pharmacology checkpoint in essentially every small-molecule discovery program, which is exactly what makes cheap, fast *in silico* hERG prediction valuable: flagging a likely liability before it costs a wet-lab assay, an animal study, or — in the cases that motivated this regulatory scrutiny in the first place — a clinical trial or a market withdrawal.

A binary blocker/non-blocker classification, which is what Section 5.5 builds, is a simplification of the property that actually matters clinically. Redfern et al. (2003), analyzing hERG/ I_{Kr} potency against clinical torsadogenic outcomes for 100 marketed and withdrawn drugs, found that risk correlates far better with the **ratio** of hERG IC₅₀ to a drug’s effective free therapeutic plasma concentration than with hERG IC₅₀ in isolation — drugs later withdrawn for torsade risk clustered at 0.1- to 31-fold separation from their therapeutic concentration, while drugs with no reported torsade risk mostly showed greater than 30-fold separation. A compound’s hERG IC₅₀ alone, without knowing the concentration at which it will actually be dosed, is therefore an incomplete safety signal — a limitation Section 5.5 returns to directly once a concrete IC₅₀ threshold has been chosen.

5.3 Classical Machine Learning Benchmarks

Three non-linear classifiers dominate practical QSAR/bioactivity classification and are benchmarked directly in Section 5.5:

Random Forest (Breiman, 2001) trains an ensemble of decision trees, each fit on a bootstrap resample of the training data and restricted at each split to a random subset of features, then averages their predictions. The random feature subsampling is what decorrelates the trees from each other — without it, bagged trees fit on the same strongly-correlated fingerprint bits would

end up highly similar, and averaging similar models does little to reduce variance. Random Forests require almost no hyperparameter tuning to get reasonable performance and are a standard, hard-to-beat-by-accident baseline for fingerprint-based classification.

Support Vector Machines (Cortes & Vapnik, 1995) find the hyperplane that separates two classes with maximum margin, then generalize to non-linear decision boundaries via the kernel trick: instead of an explicit non-linear feature map, a kernel function K computes inner products in an implicit, higher-dimensional feature space directly. The decision function takes the form

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i K(x_i, x) + b \right)$$

summed over the training examples x_i with non-zero weight α_i (the support vectors). This chapter’s project uses the radial basis function (RBF) kernel, a standard default for fingerprint data with no prior assumption of linear separability.

XGBoost (Chen & Guestrin, 2016) is a specific, heavily engineered implementation of gradient-boosted decision trees: rather than averaging independently trained trees as Random Forest does, it fits trees sequentially, each one trained to correct the current ensemble’s residual error, with an explicit regularization term controlling tree complexity directly in the objective:

$$\mathcal{L}(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

where l is a per-example loss (log-loss for the binary classification task in Section 5.5), T is the number of leaves in tree f_k , and γ, λ penalize tree size and leaf-weight magnitude respectively — the regularization is what keeps boosting, which can overfit badly if left unchecked, competitive in practice.

All three are implemented here directly with scikit-learn (Pedregosa et al., 2011) for Random Forest and SVM, and the dedicated xgboost package for XGBoost — the same tabular-data workhorses used throughout applied ML, not chemistry-specific tools. Siramshetty et al. (2020) benchmarked exactly this kind of classical approach against modern deep learning (DNNs, RNNs) on a comparably sized ChEMBL-derived hERG dataset (~9,000 compounds) and found their best overall result came from **XGBoost with RDKit descriptors** — direct support, from the hERG literature specifically, for why the outline of this book centers XGBoost for Section 5.5’s project while still benchmarking Random Forest and SVM alongside it for comparison.

5.4 Handling Bioactivity Bias

Two distinct forms of bias affect a bioactivity classifier, and both are demonstrated with real measurements from this chapter’s own dataset in Section 5.5.

Class imbalance

Running this chapter’s cleaning pipeline (Section 5.5) against the real ChEMBL hERG dataset produces 1,307 compounds labeled blockers against 458 labeled non-blockers at the chosen threshold — a roughly 2.85:1 imbalance, not the near-even split a textbook classification example

often assumes. Part of this imbalance is a real property of hERG pharmacology (many drug-like scaffolds do block the channel at some concentration), but part of it is a data-ascertainment artifact worth naming directly: compounds are deposited in ChEMBL because someone chose to test them, and hERG counter-screening is disproportionately run on compounds already flagged, by structure or by chemical series, as plausible liabilities. A dataset built this way is not a random sample of drug-like chemical space, and its class balance should not be read as an estimate of the true prevalence of hERG blockade among arbitrary compounds — a caveat that applies to bioactivity datasets generally, not just this one.

Practically, imbalance matters because raw accuracy is easy to game: a classifier that always predicts “blocker” on this dataset would already score roughly 74% accuracy while learning nothing, which is exactly why Section 5.5 reports balanced accuracy and ROC-AUC alongside accuracy throughout — both are far less inflated by a majority-class-favoring classifier. One standard mitigation is **oversampling** the minority class in the *training* set specifically: SMOTE (Synthetic Minority Over-sampling Technique; Chawla et al., 2002) generates synthetic minority-class examples by interpolating between each minority example and its nearest minority-class neighbors in feature space, rather than naively duplicating existing rows, giving the classifier more (and more varied) minority-class signal to fit against. Applying SMOTE only to the training set — never to the held-out test set, which must stay a faithful sample of the real class distribution to give an honest performance estimate — is a correctness requirement, not a style choice, and `train_and_evaluate` in `herg_qsar.py` enforces it structurally: the split happens first, and `SMOTE().fit_resample` is only ever called on the training partition.

Run on this chapter’s scaffold-split evaluation (the harder, more realistic split — see below), SMOTE moves XGBoost’s balanced accuracy from 0.661 to 0.692 and ROC-AUC from 0.803 to 0.804, at the cost of accuracy (0.776 $\rightarrow$ 0.768), recall (0.881 $\rightarrow$ 0.836), and F1 (0.857 $\rightarrow$ 0.846). This is a genuine, unglamorous trade-off, not a universal improvement: SMOTE trained the model to pay more attention to the minority (non-blocker) class, which helped the metrics that treat both classes symmetrically and cost a small amount of majority-class performance. Whether that trade is worth taking depends on which error is more costly in context — missing a true blocker (a safety miss) is generally worse than flagging a true non-blocker unnecessarily (an efficiency cost), which is an argument for caring about class-imbalance-aware metrics in the first place, independent of whether SMOTE specifically is used.

Random split vs. scaffold split

The second, distinct source of bias is in how a dataset is partitioned into train and test sets at all — the point Chapter 4 previewed and this section now measures directly. A uniformly random split routinely puts two close analogues, sharing the same core scaffold and differing only in a substituent, on opposite sides of the train/test boundary. A model can then score well on the test set not because it generalizes to new chemical space, but because it has effectively already seen a near-twin of each test compound during training. A **scaffold split** — computed here from Bemis-Murcko scaffolds (Bemis & Murcko, 1996), following the grouping-and-greedy-fill methodology documented for exactly this purpose by Yang et al. (2019) — groups compounds by their generic ring scaffold first and assigns whole scaffold groups to train or test, so no scaffold appears on both

sides. This is a harder, but more honest, estimate of how a model performs on genuinely novel chemical series.

`herg_qsar.py` implements this directly with RDKit’s `Scaffolds.MurckoScaffold` module rather than via the `deepchem` package specifically because that dependency turned out not to be viable in this project’s environment — see Section 5.5 for exactly what happened.

Running this chapter’s full pipeline (1,765 compounds, an 80/20 split) under both strategies, for all three classifiers from Section 5.3, gives a consistent answer:

Model	Split	Accuracy	Balanced acc.	ROC-AUC
XGBoost	random	0.816	0.735	0.843
XGBoost	scaffold	0.776	0.661	0.803
Random Forest	random	0.816	0.710	0.837
Random Forest	scaffold	0.793	0.631	0.790
SVM	random	0.813	0.677	0.849
SVM	scaffold	0.793	0.615	0.798

Balanced accuracy and ROC-AUC — the two metrics least sensitive to a shift in class balance between splits — drop under scaffold split for **every one of the three models**, by 0.06–0.08 in balanced accuracy and 0.04–0.05 in ROC-AUC. That is the random split systematically overestimating real-world generalization, exactly as Chapter 4 anticipated, now with a real, reproducible number attached rather than a forward reference.

Not every individual metric moves the same direction, and the one exception is worth naming rather than smoothing over: Random Forest’s recall is *higher* under scaffold split (0.941) than random split (0.931). The likely explanation is not that scaffold splitting somehow made the task easier for this one model and metric — it is that the two splits’ test sets have slightly different blocker fractions by construction (73.9% under random split vs. 76.2% under scaffold split, an artifact of which scaffold groups happened to land in each partition this run), and recall on the majority class is sensitive to that shift in a way balanced accuracy and ROC-AUC are designed not to be. This is itself the practical lesson: any single metric, evaluated on one split, can mislead; the reliable signal here is the consistent drop across *balanced* metrics and across *all three* independently trained models, not any one number in isolation.

5.5 Hands-on Project: hERG Cardiotoxicity Classifier

The project code lives in this chapter’s folder (`ch05_qsar_admet/`) and implements the full pipeline referenced throughout this chapter: extract real hERG bioactivity data from ChEMBL, clean and label it, featurize with Chapter 2’s fingerprints, split it two ways, and train and evaluate a classifier.

On DeepChem

Chapter 4 stated that “DeepChem becomes the more direct, load-bearing tool starting in Chapter 5.” Building this chapter’s project, the specific intended use — DeepChem’s documented scaffold-splitting utility — was attempted first, and hit a real, reproducible problem: in this project’s environment, `import deepchem` unconditionally imports `deepchem.trans.transformers`, which unconditionally imports `tensorflow` at module load time, even though TensorFlow is not declared as an installation dependency and nothing in this project’s use case (a scaffold splitter) requires it. Installing a multi-gigabyte deep learning framework as a transitive, undeclared dependency of one utility function is a poor trade for a book whose own stated engineering principle (Chapter 4, Section 4.2) is using the most pedagogically direct tool for the job. Section 5.4’s scaffold split is therefore implemented directly against RDKit’s own `Scaffolds.MurckoScaffold` module — a well-documented, roughly twenty-line algorithm — rather than through DeepChem. This is a smaller-scope fulfillment of Chapter 4’s forward reference than “DeepChem becomes load-bearing,” and that gap is recorded here rather than papered over.

Extract and label

`herg_qsar.py` extracts IC50 bioactivity records for hERG (ChEMBL240) from the ChEMBL API, paginating exactly as Chapter 4’s `extract_bioactivities` does. Each record is standardized with Chapter 4’s technique (Section 4.1: largest fragment, uncharge, canonical tautomer) and duplicate measurements of the same standardized structure are merged via the median, as in Chapter 4. Each resulting compound is then labeled by a fixed threshold on its median IC50:

```
DEFAULT_THRESHOLD_NM = 10_000.0 # 10 uM
is_blocker = median_ic50_nm <= DEFAULT_THRESHOLD_NM
```

This 10 μ M cutoff is a deliberate, explicit modeling choice for this project, not a value taken from a specific external paper’s stated methodology — that distinction matters, because the exact numeric threshold used for hERG “blocker” classification genuinely varies across the published literature, and claiming otherwise would misattribute a number this chapter chose. It is chosen as a round order-of-magnitude value broadly consistent with the qualitative safety-margin reasoning in Redfern et al. (2003, Section 5.2): this project’s own dataset has a median IC50 of 3.5 μ M, so 10 μ M sits at roughly the upper-middle of the observed potency range rather than at an extreme. Changing it is a one-line edit (`--threshold-nm`), and doing so changes both the label balance and, materially, how hard the resulting classification task is — worth trying directly rather than taking this chapter’s specific choice as definitive.

Featurize, split, train

Compounds are featurized with the same Morgan/ECFP4 fingerprint (`radius=2`, `fpSize=2048`) as Chapter 2, split with either `random_split` or `scaffold_split` (Section 5.4), and passed to one of the three classifiers from Section 5.3 via a single shared interface:

```
def train_and_evaluate(records, model_type="xgboost", split_type="scaffold",
                        use_smote=False, seed=0) -> dict:
    ...
```

so the same clean/featurize pipeline underlies every combination of model, split, and oversampling choice reported in Sections 5.3–5.4.

Running it and reading the output

```
cd ch05_qsar_admet
pip install -r requirements.txt
python herg_qsar.py --model xgboost --split both
```

Running the default configuration against the live ChEMBL API (or offline, deterministically, with `--use-cached-raw`) extracts 3,000 raw IC50 records, cleans them to 1,765 unique standardized compounds (1,307 blockers / 458 non-blockers at the 10 μ M threshold), and reports, for both the random and scaffold split, exactly the metrics tabulated in Section 5.4 – XGBoost’s scaffold-split ROC-AUC of 0.803 against its random-split ROC-AUC of 0.843 is the single number this chapter’s narrative has been building toward since Chapter 4.

Reproducibility

Dependencies are version-floored (`rdkit>=2023.9.1`, `scikit-learn>=1.3`, `xgboost>=2.0`, `imbalanced-learn>=0.11` in `requirements.txt`, validated against 2026-era releases of each). `data/raw_herg_bioactivities_sample.json` bundles a real 3,000-record extract (fetched 2026-08-19) so the full clean $\rightarrow$ featurize $\rightarrow$ split $\rightarrow$ train $\rightarrow$ evaluate pipeline runs offline and deterministically with `--use-cached-raw`, following Chapter 4’s resilience pattern directly (the live ChEMBL connection did in fact time out once while this fixture was being prepared – an unprompted, real-world demonstration of exactly the failure mode that pattern exists to survive). The 28-test suite in `tests/test_herg_qsar.py` checks extraction, cleaning, labeling, featurization, both split strategies (including an explicit check that scaffold split never places the same scaffold in both train and test), and end-to-end training against both synthetic edge cases and a real slice of the bundled fixture. `pip install -r requirements-dev.txt && pytest` reproduces all 28 results.

Limitations and what comes next

Everything in this chapter operates on the same 2D representation Chapter 2 introduced, with the same blind spot flagged there: a fingerprint encodes molecular constitution, not the 3D shape a compound actually presents to hERG’s binding pocket. Sanguinetti and Tristani-Firouzi’s point about hERG specifically (Section 5.2) – that its unusually large, aromatic-lined pore accommodates structurally diverse binders through more than one plausible binding mode – is a concrete reason this class of liability may be harder for a fixed 2D fingerprint to fully capture than a more geometrically constrained target would be; Chapter 6’s graph neural networks (learning a representation directly from structure rather than fixing one in advance) and Chapter 11’s docking methods (explicit 3D binding-pose prediction) are both direct responses to this same limitation, applied to different parts of the pipeline. Separately, a fixed IC50 threshold discards the concentration-dependent safety-margin reasoning Redfern et al. established (Section 5.2) – a regression model predicting IC50 directly, combined with a compound’s anticipated therapeutic concentration, is a more complete safety assessment than any single blocker/non-blocker classifier. And a scaffold split, while a substantially more honest estimate of generalization than a

random split, is still evaluated against scaffolds already present in a fixed historical dataset — it is not a test against chemistry that does not yet exist, which is precisely the harder problem Chapter 7’s generative models are evaluated against.

A note on Google Colab

rdkit, xgboost, and imbalanced-learn are not preinstalled on Colab’s default runtime (scikit-learn, numpy, and requests are); run `!pip install rdkit xgboost imbalanced-learn` in the first cell. No GPU is required — training any of the three classifiers on ~1,400 compounds’ worth of 2048-bit fingerprints completes in well under a minute on CPU.

References

- Van de Waterbeemd, H., & Gifford, E. (2003). ADMET in silico modelling: towards prediction paradise? *Nature Reviews Drug Discovery*, 2(3), 192–204. <https://doi.org/10.1038/nrd1032>
- Sanguinetti, M. C., & Tristani-Firouzi, M. (2006). hERG potassium channels and cardiac arrhythmia. *Nature*, 440(7083), 463–469. <https://doi.org/10.1038/nature04710>
- Redfern, W. S., Carlsson, L., Davis, A. S., Lynch, W. G., MacKenzie, I., Palethorpe, S., Siegl, P. K., Strang, I., Sullivan, A. T., Wallis, R., Camm, A. J., & Hammond, T. G. (2003). Relationships between preclinical cardiac electrophysiology, clinical QT interval prolongation and torsade de pointes for a broad range of drugs: evidence for a provisional safety margin in drug development. *Cardiovascular Research*, 58(1), 32–45. [https://doi.org/10.1016/s0008-6363\(02\)00846-5](https://doi.org/10.1016/s0008-6363(02)00846-5)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Siramshetty, V. B., Nguyen, D. T., Martinez, N. J., Southall, N. T., Simeonov, A., & Zakharov, A. V. (2020). Critical assessment of artificial intelligence methods for prediction of hERG channel inhibition in the “Big Data” era. *Journal of Chemical Information and Modeling*, 60(12), 6007–6019. <https://doi.org/10.1021/acs.jcim.0c00884>
- Bemis, G. W., & Murcko, M. A. (1996). The properties of known drugs, part 1: Molecular frameworks. *Journal of Medicinal Chemistry*, 39(15), 2887–2893. <https://doi.org/10.1021/jm9602928>
- Yang, K., Swanson, K., Jin, W., Coley, C., Eiden, P., Gao, H., Guzman-Perez, A., Hopper, T., Kelley, B., Mathea, M., Palmer, A., Settels, V., Jaakkola, T., Jensen, K., & Barzilay, R. (2019). Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8), 3370–3388. <https://doi.org/10.1021/acs.jcim.9b00237>

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

See Chapter 1’s references for Mendez et al. (2019, ChEMBL) and Chapter 4’s references for Ramsundar et al. (2019, DeepChem’s own documented citation) — both reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers’ recommended citation is “RDKit: Open-source cheminformatics. <https://www.rdkit.org>” (confirmed directly from the project’s own documentation), matching the convention established in Chapter 4’s references. The Pedregosa et al. (2011) scikit-learn citation is reproduced from the paper’s own metadata on jmlr.org, which lists no DOI for this article.

All dataset sizes, class balances, and model metrics cited in Sections 5.4–5.5 were computed directly by running `herg_qsar.py` against the live ChEMBL API on 2026-08-19, not taken from a secondary source — see `data/raw_herg_bioactivities_sample.json` and `herg_qsar.py` to reproduce.